

Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas
Sistemas Operativos II
Ingeniero Luis Fernando Alvarado

**Sistema de Registro de Puntos de Interés con Base
de Datos Geoespacial**

Bryann Jose Alvarez Osorio 7691-22-7641
Guatemala 22/04/2026

Introducción

El presente proyecto consiste en el desarrollo de una aplicación web contenerizada para registrar, almacenar y consultar puntos de interés geográficos. La solución fue diseñada para ejecutarse completamente en un entorno local mediante Docker Compose, sin depender de servicios externos y siguiendo una arquitectura basada en contenedores independientes.

El sistema permite gestionar puntos de interés como sitios culturales, naturales, gastronómicos, entre otros. Para ello, almacena información básica como el nombre, la descripción, la categoría y las coordenadas geográficas de cada punto.

Además, la aplicación incluye consultas geoespaciales que permiten buscar puntos cercanos a una ubicación específica dentro de un radio determinado.

La solución fue desarrollada utilizando PostgreSQL con PostGIS como base de datos geoespacial, Node.js con Express como servidor de aplicaciones y Nginx como proxy reverso y punto de entrada único. Todo el sistema se despliega mediante Docker Compose, utilizando una red interna y volúmenes persistentes para garantizar el almacenamiento de la información.

Objetivo del proyecto

El objetivo principal del proyecto fue implementar una aplicación web geoespacial contenerizada que permitiera cumplir con los siguientes requerimientos:

- registrar puntos de interés con nombre, descripción, categoría, latitud y longitud
- listar todos los puntos registrados
- filtrar puntos por categoría
- buscar puntos cercanos a una coordenada dada usando un radio de búsqueda
- cargar automáticamente datos iniciales al levantar el sistema por primera vez
- permitir acceso desde navegador a través de un proxy web
- ejecutar todos los componentes en contenedores independientes conectados por una red Docker interna.

Descripción general del sistema

El sistema está compuesto por tres servicios principales:

1. Base de datos geoespacial

Se utilizó PostgreSQL con la extensión PostGIS para almacenar los puntos de interés y realizar operaciones espaciales. La tabla principal almacena nombre, descripción, categoría, coordenadas y una columna geoespacial `GEOGRAPHY(POINT, 4326)`, lo que permite ejecutar búsquedas por distancia y proximidad.

2. Backend o servidor de aplicaciones

Se desarrolló un backend en Node.js con Express encargado de exponer una API REST para consultar, registrar y actualizar los puntos de interés. También administra la carga de imágenes asociadas a cada punto y las expone mediante una ruta pública de archivos.

3. Proxy web reverso

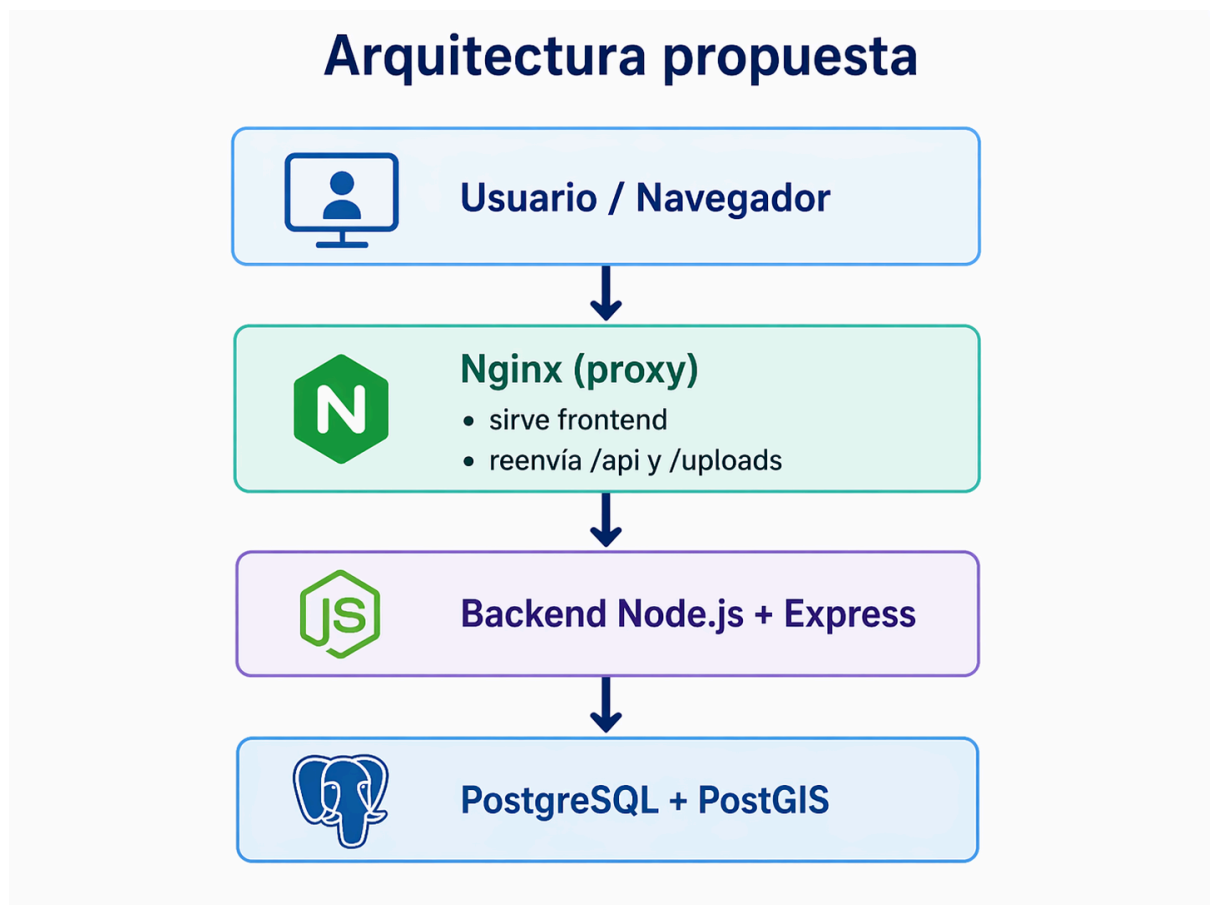
Se implementó Nginx como punto de entrada único al sistema. Este servicio sirve el frontend estático y reenvía las solicitudes `/api/` y `/uploads/` hacia el backend.

Arquitectura del sistema

La arquitectura del proyecto se encuentra dividida en tres contenedores independientes:

- db: contenedor de base de datos con PostGIS
- backend: contenedor del servidor API en Node.js/Express
- proxy: contenedor Nginx para servir el frontend y actuar como proxy reverso.

Todos estos servicios son orquestados con Docker Compose. Además, se define explícitamente una red llamada `geo_network`, lo que permite la comunicación entre contenedores sin utilizar la red bridge por defecto. También se define un volumen llamado `postgres_data` para conservar la información de la base de datos.



Justificación de las tecnologías elegidas

Para este proyecto se seleccionaron tecnologías que permitieran cumplir de manera clara y funcional con los requerimientos planteados.

PostgreSQL + PostGIS

Se eligió PostgreSQL como sistema gestor de base de datos por su robustez y compatibilidad con entornos contenerizados. Sobre esta base se utilizó PostGIS, ya que permite almacenar coordenadas geográficas y ejecutar operaciones espaciales avanzadas como medición de distancias y búsquedas por radio. Esto resultó ideal para implementar la funcionalidad de puntos cercanos.

Node.js + Express

Se eligió Node.js con Express por su simplicidad, rapidez de desarrollo y facilidad para construir una API REST ligera. Esta combinación permitió implementar las rutas necesarias para listar puntos, filtrarlos por categoría, consultarlos por cercanía, crear nuevos registros y actualizarlos.

Nginx

Se utilizó Nginx como proxy reverso por ser una solución estable, ligera y ampliamente utilizada. Su uso permitió centralizar el acceso al sistema en una sola URL local.

Docker Compose

Se utilizó Docker Compose para orquestar todos los servicios del proyecto, permitiendo levantar la solución completa con un solo comando y manteniendo la configuración organizada. Gracias a esto, el entorno puede reproducirse fácilmente en otra máquina.

Frontend con HTML, CSS y JavaScript

Se decidió usar un frontend web simple basado en HTML, CSS y JavaScript, lo cual fue suficiente para construir una interfaz clara y funcional. Para la visualización geográfica se utilizó Leaflet con mapas de OpenStreetMap, permitiendo mostrar los puntos sobre un mapa interactivo.

Persistencia de datos

La persistencia del sistema se garantiza principalmente mediante el volumen Docker `postgres_data`, que se encuentra montado en la ruta `/var/lib/postgresql/data` del contenedor de base de datos. Esto permite que la información almacenada en PostgreSQL se conserve incluso si los contenedores son detenidos, reiniciados o eliminados.

Adicionalmente, el backend utiliza un montaje de la carpeta `./backend/uploads` hacia `/app/uploads` dentro del contenedor, con el objetivo de conservar las imágenes subidas por los usuarios. De esta manera, los archivos cargados no se pierden al reconstruir el contenedor del backend.

La carga inicial de la base de datos se realiza mediante scripts SQL ubicados en `db/init/`, los cuales se ejecutan automáticamente la primera vez que se crea el contenedor de la base de datos. Estos scripts crean las extensiones espaciales, el esquema de tablas y cargan al menos cinco puntos de ejemplo.

Comunicación entre servicios

La comunicación entre los servicios se realiza a través de la red Docker interna `geo_network`, definida explícitamente en el archivo `docker-compose.yml`. Gracias a esta red, cada contenedor puede comunicarse con los demás utilizando el nombre del servicio como host.

En este proyecto:

- el backend se conecta a la base de datos usando el host `db`
- el proxy Nginx se conecta al backend usando el host `backend`
- el usuario accede únicamente al proxy a través de `http://localhost`.

Nginx redirige las rutas `/api/` y `/uploads/` hacia el backend, lo que permite separar claramente las responsabilidades de cada contenedor y mantener una entrada única al sistema.

Funcionalidades implementadas

El sistema desarrollado implementa las siguientes funcionalidades:

Registro de puntos de interés

Se pueden registrar nuevos puntos de interés enviando nombre, descripción, categoría, latitud, longitud e imágenes. El backend valida los campos obligatorios y guarda la ubicación tanto en coordenadas numéricas como en formato geoespacial.

Listado de puntos

La API permite listar todos los puntos registrados, devolviendo también la información de sus imágenes asociadas.

Filtrado por categoría

Existe una ruta específica para consultar únicamente los puntos pertenecientes a una categoría determinada.

Búsqueda por cercanía

Se implementó una búsqueda geoespacial mediante coordenadas y radio, utilizando funciones de PostGIS como ST_DWithin y ST_Distance. Esto permite localizar puntos cercanos a una posición específica.

Actualización de puntos

El sistema permite editar registros existentes, modificar datos generales del punto, agregar nuevas imágenes y eliminar imágenes previas.

Visualización en mapa

Desde el frontend, los puntos se muestran en un mapa interactivo. También se pueden visualizar detalles, realizar búsquedas, usar la ubicación actual del usuario y trazar una ruta hacia un punto seleccionado.

Carga inicial de ejemplo

La base de datos carga automáticamente cinco puntos de interés de ejemplo al inicializarse por primera vez, lo cual cumple con uno de los requisitos funcionales obligatorios de la tarea.

Dificultades encontradas y soluciones aplicadas

Durante el desarrollo del proyecto se presentaron varias dificultades técnicas, las cuales fueron resueltas de la siguiente manera:

Problema de conexión entre backend y base de datos

Inicialmente, la conexión fallaba porque el backend intentaba conectarse a la base de datos usando `localhost`. En un entorno Docker esto no funciona entre contenedores independientes. La solución fue utilizar el nombre del servicio db como host de conexión dentro de la red interna.

Arranque prematuro del backend

Otra dificultad fue que el backend podría iniciar antes de que la base de datos estuviera completamente lista. Para resolverlo, se agregó un `healthcheck` al servicio db y se configuró `depends_on` con condición de servicio saludable para el backend.

Acceso del frontend al backend

En una etapa inicial, el frontend consumía directamente el backend por puerto. Esto no cumplía con el requisito de entrada única por proxy. Se ajustó la configuración para que el frontend utilizara rutas relativas como `/api`, mientras que Nginx se encargó del reenvío hacia el backend.

Manejo de imágenes en edición

Se presentó el reto de permitir que un punto pudiera editarse sin perder el control del número máximo de imágenes. Para ello se implementó lógica que permite marcar imágenes existentes para eliminar y luego agregar nuevas, manteniendo un máximo de cinco por punto.

Carga inicial de datos y estructura geoespacial

Fue necesario separar la inicialización de la base en varios scripts SQL: uno para extensiones, otro para esquema y otro para datos de prueba. Esto permitió que la base se construyera de forma ordenada y automática al levantarse por primera vez.

Buenas prácticas aplicadas

Durante el desarrollo del proyecto se aplicaron varias buenas prácticas técnicas:

- separación de servicios por contenedor
- uso de variables de entorno para configuración
- definición explícita de red Docker interna
- persistencia de datos mediante volúmenes
- uso de proxy reverso como punto de entrada único
- separación entre frontend, backend y base de datos
- validaciones básicas en backend para datos obligatorios y tipos numéricos
- restricción del tipo y tamaño de imágenes cargadas.

Conclusión

Se logró desarrollar una aplicación web geoespacial funcional y completamente contenerizada, cumpliendo con los requerimientos principales de la tarea. La solución implementa una arquitectura basada en contenedores independientes para base de datos, backend y proxy reverso, todos orquestados mediante Docker Compose.

El sistema permite registrar, consultar, filtrar y buscar puntos de interés geográficos de forma eficiente, además de cargar datos iniciales automáticamente y mantener persistencia mediante volúmenes Docker. La integración de PostgreSQL con PostGIS permitió resolver correctamente las necesidades geoespaciales del proyecto, mientras que Nginx garantiza un acceso unificado desde el navegador.

En conclusión, el proyecto cumple con los objetivos funcionales y técnicos planteados, y representa una solución reproducible, organizada y adecuada para el escenario solicitado en la tarea.

Link de repositorio: <https://github.com/BAlvarez12/proyecto-geoespacial.git>